

Explaining the Behavior of an Instruction-Driven Research Agent: A Pilot Study

Christian Block

July 2026

Abstract

A language model can be set up to run a small research loop on its own: given a text file of instructions and a way to run experiments, it repeatedly proposes something to try, runs it, looks at the result, and decides what to do next. Change the instructions and the loop behaves differently, but it is usually unclear which part of the instructions is responsible for which part of the difference. This report treats the instruction file as a small configuration, in the same way an algorithm’s hyperparameters form a configuration, and borrows a standard technique from automated machine learning – ablation studies over a fractional factorial design – to measure the effect of each instructional choice on the loop’s behavior. Five instructional choices were varied (how precisely the evaluation metric is defined, whether the agent may compare two model families or must stick to one, whether its experiment budget is fixed or adaptive, whether it must explain its reasoning, and whether it favors broad exploration or immediate refinement), an eight-configuration ablation design was run against a real machine learning task (tuning Random Forest and Gradient Boosting classifiers), and a simple statistical model was fit to explain which instructional choice drives which aspect of the resulting behavior. The method correctly recovers an expected relationship (the instruction to write full explanations is, unsurprisingly, the dominant driver of how much text the agent produces) and also finds a less obvious one: whether the agent is told to explore broadly or to refine immediately turns out to be the strongest single predictor of the accuracy it eventually reaches. A held-out prediction, made before the corresponding configuration was ever run, is checked against a real execution: one quantity is predicted almost exactly, another is off by a small but informative amount. All numbers in this report come from real, executed scikit-learn experiments; nothing is simulated. The report closes with an honest account of what this pilot could not do – in particular, that the language-model role in every run was played by the author rather than by an independent model call – and what a larger follow-up would need to change.

1 Introduction

Suppose you want a language model – a program trained on large amounts of text that, given a prompt, produces further text – to act as a small autonomous researcher. You give it a description of a task, for example “find a good Random Forest configuration for this dataset”, together with a tool it can call to actually run an experiment and get a number back. The model reads the instructions, proposes something to try, calls the tool, reads the result, and decides what to try next, repeating this until it decides to stop. Systems built this way are sometimes called autoresearch loops, and they are attractive precisely because they need so little supervision: point one at a problem, and it works through the same cycle a human researcher would – hypothesize, run an experiment, interpret, decide – without a person in the loop.

The entire behavior of such a system, however, is steered by one artifact: the instruction file that starts it off. We refer to this file throughout as `program.md`. Anyone who has actually run one of these loops more than a few times notices something quickly: small changes to the wording of the instructions produce noticeably different research behavior. Tell the agent to stop after a fixed number of experiments

and it stops after a fixed number of experiments; phrase the stopping rule differently and it might run twice as many, or half as many. Ask for a precise evaluation metric and it reasons about numbers differently than if the metric is left vague. This is not surprising in itself. What is unsatisfying is that the relationship is usually left as an observation rather than an explanation: we notice that the instructions matter, but we do not have a way of saying which specific part of the instructions is responsible for which specific part of the resulting behavior, nor by how much.

This gap matters because it prevents two things a scientist normally wants from an experimental result. First, it prevents prediction: without knowing which instructional choice drives which behavior, we cannot say in advance what will happen if we try a combination of instructions we have not yet tried. Second, it prevents explanation to someone else: "the agent behaved differently because I changed the instructions" is not a statement anyone else can act on, whereas "the agent ran fewer experiments because the stopping rule was stricter, and this accounts for about half of the variation we saw" is.

This project treats that gap as a measurement problem rather than a mystery, and borrows the tools for it from a field that already solved a structurally identical problem: automated machine learning. When a learning algorithm has several hyperparameters, researchers do not simply observe that changing hyperparameters changes performance and leave it there. They routinely apply techniques – ablation analysis, fractional factorial experiment design, and variance decomposition over hyperparameters – to work out which hyperparameters matter, how much each one matters, and whether they interact [1, 2, 3]. Our central observation is that an instruction file like `program.md` is, structurally, no different from a hyperparameter vector: it is a set of discrete, largely independent choices that jointly determine a system's behavior. If that observation is right, the same tools should transfer directly, and that transfer is what this report tries out.

To keep the experiment small enough to actually run and check by hand, the domain was restricted to a single, well-understood machine learning task: tuning Random Forest and Gradient Boosting classifiers on one fixed, small benchmark dataset. Nothing about this report is trying to show that these particular models can be tuned well – that is a solved problem. The point of choosing a simple, familiar task is that it lets us focus entirely on the question that actually matters here: given a fixed task, does changing the instructions change the research *process* in a way we can measure, attribute, and predict?

The rest of this report is organized as follows. Section 2 briefly explains the three ideas borrowed from automated machine learning, for readers who have not seen them before. Section 3 walks through the six steps of the research plan that was executed, one by one. Section 4 states the exact experimental setup. Section 5 presents and discusses the results. Section 6 is a frank account of what this pilot did not manage to do and why. Section 7 concludes.

2 Background

Three ideas underlie this project, all originally developed to make sense of algorithms with many hyperparameters rather than instruction files, but they transfer without modification.

The first is *variance decomposition over configuration*. If an algorithm has several hyperparameters and its performance varies as we change them, a natural question is how much of that variation each hyperparameter is individually responsible for. Functional ANOVA [2] answers this by treating the outcome (say, validation accuracy) as a function of the hyperparameter vector and decomposing its variance into a part attributable to each hyperparameter on its own, plus parts attributable to interactions between pairs of hyperparameters. The result is a short list of interpretable numbers – "hyperparameter A explains 60% of the variance, hyperparameter B explains 5%" – instead of a black-box performance curve. This report reuses exactly this idea, with axes of an instruction file standing in for hyperparameters.

The second is *ablation analysis*, which asks a related but distinct question: given two specific configurations with different performance, which of the individual differences between them is responsible for

how much of the gap [1]. This complements variance decomposition by giving a step-by-step account of one specific comparison, rather than a global summary across many configurations.

The third is a practical constraint that both of the above run into: measuring the effect of k two-level choices in full requires testing all 2^k combinations, which becomes expensive quickly. *Fractional factorial design* sidesteps this by testing a carefully chosen subset of combinations, small enough to be affordable, while still guaranteeing that the individual (“main”) effect of every choice can be estimated without bias from the others. The classical construction for this, due to Plackett and Burman [3], produces such a subset in multiples of four runs – eight runs suffice for up to seven two-level choices – and is standard practice in industrial experimentation. The one thing it gives up, in exchange for needing so few runs, is the ability to tell apart interaction effects between pairs of choices; we return to why this matters in Section 5.

Put together, these three ideas describe a small pipeline: pick a manageable subset of configurations to actually run (fractional factorial design), fit a simple model that attributes the resulting variation in behavior to individual configuration choices (variance decomposition), and use that model to reason about specific comparisons (ablation-style reasoning). This is exactly the pipeline applied below to `program.md`.

3 Method

The project follows a six-step plan. Steps 2 through 6 were carried out in full for this pilot, using real, executed machine learning experiments throughout; Step 1 was deliberately left for a follow-up, for reasons explained below. The overall flow is: turn the instruction file into a configuration (Step 2); decide which configurations to actually run (Step 3); run them against a real experiment executor (also part of Step 3, described together with the loop mechanics); turn each run’s record into a handful of numbers (Step 4); fit a model relating configuration to those numbers (Step 5); and check that model’s predictions against a real, held-out run (Step 6).

3.1 Step 1: a human reference trace

The original plan for this project called for a human researcher to work through one concrete instance of the same kind of problem the agent is given – for example, “does the feature-subsampling ratio affect generalization of a Random Forest on this dataset” – while recording their screen and thinking aloud, and for the resulting recording to be transcribed and split into a sequence of steps using a small fixed vocabulary: hypothesize, design an experiment, run it, observe the result, interpret it, decide what to do next. Such a trace would give a genuine point of comparison: instead of only asking whether the agent’s behavior changes with the instructions, one could also ask how close the agent’s research process comes to how a person would actually go about the same problem.

Producing this trace requires an actual person to sit down and do the task under recording; it is not something that can be substituted computationally, and it was set aside for this pilot by explicit agreement, to be picked up as the first item of follow-up work (Section 6). One consequence of planning ahead for this step, however, shaped everything that follows: the six-item action vocabulary meant for the human trace was adopted as the logging format for every automated run as well, so that once a human trace does exist, comparing it to any run already collected requires no change to the rest of the pipeline.

3.2 Step 2: turning the instruction file into a configuration

The first executed step reduces the free-text instruction file to a small, structured set of choices, the same way a hyperparameter vector reduces an algorithm to something whose behavior can be studied systematically rather than described anecdotally. Five instructional choices were identified by reading

through example instruction files and noting which sentences plausibly influence behavior; each choice was restricted to exactly two levels, giving a five-bit configuration.

The metric axis controls how precisely the instructions define success: at one level the file simply says to "evaluate model quality in whatever way you judge appropriate"; at the other it specifies exactly what to compute – mean five-fold cross-validated accuracy together with its standard deviation across folds, with the standard deviation used only to break close ties. The breadth axis controls whether the agent must commit to a single model family, Random Forest or Gradient Boosting, for an entire run, or is free to try both. The stopping-rule axis controls the experiment budget: either a fixed budget of exactly three experiments, or an adaptive rule that allows up to eight experiments but stops as soon as one experiment fails to improve on the best result so far by more than a small threshold. The output-format axis controls whether responses must be terse and numeric, or must include full, reasoned explanations of hypotheses and interpretations. The emphasis axis controls search strategy: whether the agent should prioritize trying several different configurations before refining any one of them, or should immediately refine the first promising result it finds with nearby variations.

A configuration is then simply a vector of five zeros and ones, for instance (1, 0, 0, 1, 1), one bit per axis in the order metric, breadth, stopping rule, output format, emphasis. Rather than writing out each of the 32 possible instruction files by hand, a single template with a placeholder for each axis is filled in automatically from a configuration vector by a short script. This guarantees that any two generated instruction files differ only in the axes that were actually varied between them – a property the later steps depend on, since attributing a behavioral difference to a specific axis only makes sense if nothing else changed at the same time.

3.3 Step 3: choosing which configurations to run, and running them

With five two-level axes, testing every combination means 32 configurations, and because a research loop is not perfectly deterministic, each configuration would ideally be repeated several times to separate a genuine effect of the configuration from ordinary run-to-run variation. Running all 32 configurations with several repeats each was outside the scope of what a single pilot could cover, so an eight-configuration Plackett–Burman design was used instead: a specific, non-arbitrary subset of the 32 possible configurations, generated by the classical cyclic construction (a base pattern of +, +, +, −, +, −, − cyclically shifted seven times, plus one row of all minus signs), chosen so that the individual effect of every one of the five axes can still be estimated without being distorted by any of the others. The price of using only eight runs is that interactions between pairs of axes – for instance, whether the effect of the emphasis axis depends on the breadth axis – cannot be told apart from each other; this is a standard and explicitly accepted trade-off, and Section 5 shows a concrete case where it matters. Table 1 lists the resulting eight configurations.

Table 1: The eight configurations used in the ablation study. Columns are the five axes from Section 3.2 in order: metric, breadth, stopping rule, output format, emphasis.

Configuration	Metric	Breadth	Stopping	Output	Emphasis
0	1	1	1	0	1
1	0	1	1	1	0
2	0	0	1	1	1
3	1	0	0	1	1
4	0	1	0	0	1
5	1	0	1	0	0
6	1	1	0	1	0
7	0	0	0	0	0

Each configuration was run twice, with two different random seeds, giving sixteen runs in total. A larger

study would use three to five repeats per configuration to separate genuine configuration effects from noise more reliably; two were used here to keep the pilot within reach of a single sitting, and this is listed plainly as a limitation in Section 6.

Running a configuration means generating its instruction file and then executing the loop it describes against a real machine learning task. Every run addresses the same question – how does hyperparameter choice affect the generalization accuracy of Random Forest and Gradient Boosting classifiers on the Breast Cancer Wisconsin diagnostic dataset, a small, standard benchmark bundled with scikit-learn – using the same fixed 70/30 split of the data into a training and a validation portion, so that only the instructions and the loop’s own randomness differ from one run to the next. At each step of a run, the agent can request one experiment by naming a model family and a set of hyperparameters; an ordinary piece of Python code, not a language model, then actually fits that model, computes five-fold cross-validated accuracy and its standard deviation on the training portion, and computes accuracy on the held-out validation portion, and reports these real numbers back. This executor is the one part of the loop that never involves a language model: every number the agent sees is a genuine measurement from an actually-trained scikit-learn model. The agent, in turn, responds at each step with one of two structured actions: propose an experiment with a stated hypothesis, or interpret the last result and decide whether to continue or stop, giving a final recommendation if stopping. Logging every action this way, rather than as free-flowing prose, is what makes the next step mechanical rather than a difficult natural-language-understanding problem.

3.4 Step 4: turning a run into a handful of numbers

Because each run already produces a structured, itemized record rather than a wall of prose, reducing it to a row of numbers is a matter of straightforward parsing rather than interpretation. Five quantities are computed per run: the number of experiments actually executed before the agent stopped; the number of distinct model families tried, which is one or two depending on whether the agent ever switched between Random Forest and Gradient Boosting; the spread (largest minus smallest) of the tree-count hyperparameter across the experiments tried in a run, used as a simple stand-in for how widely the agent searched; the best cross-validated accuracy reached across all experiments in the run, standing in for the quality of the research outcome; and the average length, in characters, of the agent’s stated hypotheses, used as a crude proxy for verbosity. The result of this step is a single table with one row per run: the five-bit configuration, which repeat it was, and these five numbers.

3.5 Step 5: a simple model of configuration and behavior

For each of the five behavioral quantities, a linear model was fit relating it to the five configuration axes, each axis coded as -1 or $+1$ rather than 0 or 1 . Coding the axes this way, together with the fact that the Plackett–Burman design is orthogonal – meaning the coded columns for any two axes are uncorrelated across the eight configurations – has a useful consequence: each axis’s coefficient can be estimated independently of the others, and the share of the model’s total explained variance attributable to a given axis is simply that axis’s squared coefficient divided by the sum of all five squared coefficients. This turns the fitted model into a small, readable table: for each behavioral quantity, which axis matters most, and what fraction of the variation it accounts for. As a cross-check, a shallow decision tree (limited to three levels) was also fit for each quantity, to confirm it splits on the same axes the linear model considers important. Before fitting either model, the two repeats of each configuration were averaged together, so the model was fit on eight configuration-level values rather than sixteen individual runs, matching the resolution the ablation design was actually built to support.

3.6 Step 6: checking the model with a real intervention

A model that only fits the data used to build it has not yet demonstrated anything beyond correlation. The last step is meant to turn the fitted model into a claim that can actually be tested: one configuration,

$(0, 0, 0, 0, 1)$, was chosen because it was not among the eight used to fit the model in Step 5. Using the fitted model, two of its behavioral quantities were predicted *before* this configuration was ever run. The corresponding instruction file was then generated and executed twice, using two random seeds not used anywhere else in this pilot, and the predictions were compared against what actually happened. This is the difference between saying "the model fits the data it was trained on" and saying "the model's implied claim – if you set the instructions this way, you should get approximately this behavior – held up when we actually tried it."

4 Experimental Setup

Every run used the Breast Cancer Wisconsin diagnostic dataset as bundled with scikit-learn: 569 samples, 30 numeric features, a binary classification target. A single 70/30 train/validation split, fixed with the same random seed across every run, ensured that only the instructions and the loop's own stochastic behavior varied between configurations. The two model families available to the agent were scikit-learn's `RandomForestClassifier` and `GradientBoostingClassifier`; for the former the agent could vary the number of trees, tree depth, minimum leaf size, and the fraction of features considered at each split, and for the latter the number of boosting stages, tree depth, learning rate, and subsample fraction. Regardless of what a given configuration's instructions emphasized, the executor always reported the same measurements back: five-fold cross-validated accuracy with its standard deviation on the training split, and accuracy on the held-out validation split.

In total, the pilot consisted of eight ablation configurations run twice each, plus one held-out configuration also run twice, for eighteen runs and forty-six individual model-fitting experiments across all of them combined. All code, generated instruction files, and raw run records are kept alongside this report so the pipeline can be re-run or inspected directly.

5 Results

Table 2 summarizes, for each of the five behavioral quantities from Step 4, which axis the fitted linear model identifies as the dominant driver, how large a share of the variation it accounts for, the direction of the effect, and the model's overall fit (R^2) across the eight configuration-level values.

Table 2: Which axis explains which behavioral quantity, according to the fitted linear model.

Behavioral quantity	Dominant axis	Variance share	Direction of effect	R^2
Verbosity (avg. proposal length)	Output format	74%	full explanations $\rightarrow$ roughly 2–4 $\times$ longer proposals	0.96
Model families tried	Breadth, output, emphasis (tied)	33% each	broad $\uparrow$, verbose $\uparrow$, exploit-first $\downarrow$	1.00
Best accuracy reached	Emphasis	84%	exploit-first $\rightarrow$ +0.0025 mean accuracy	1.00
Number of experiments	Stopping, emphasis (tied)	50% each	adaptive stopping $\downarrow$, exploit-first $\uparrow$	0.67
Search width	Emphasis	49%	exploit-first $\rightarrow$ wider spread	0.83

The first row is, by design, close to a sanity check. The output-format axis directly instructs the agent to write either terse or fully reasoned responses, so finding that it explains 74% of the variation in how long the agent's stated hypotheses are, with a statistically significant coefficient, is exactly what should happen if the whole pipeline – from generating the instruction file, through running the loop, logging its

steps, parsing them into numbers, and fitting a model – is working correctly end to end. If some unrelated axis had turned out to dominate verbosity instead, that would have pointed to a bug somewhere in the pipeline rather than a genuine finding. Because this check passes cleanly, it is reasonable to give some credence to the less obvious result that follows.

That result concerns the emphasis axis, which says nothing about accuracy at all – it only tells the agent whether to explore several configurations before refining any one of them, or to refine the first promising result immediately. Despite that, it accounts for 84% of the variation in the best accuracy reached across the eight configurations, with configurations that favor immediate refinement outperforming those that favor broad exploration first by about a quarter of an accuracy percentage point on average. This is a small effect in absolute terms, but a highly consistent one across the design, and it has a plausible explanation: with only two to four experiments available in most of these runs, a strategy that commits early to refining a promising region of the hyperparameter space has less room to waste an experiment exploring a probably-worse region than a strategy that insists on breadth first. It is exactly the kind of effect that would be invisible without decomposing the instruction file into separate axes and measuring each one’s contribution directly – one would otherwise only observe that some instruction files happen to lead to better results than others, without being able to say why.

The remaining rows are more mechanical but still informative. The breadth axis, unsurprisingly, is one of three tied drivers of how many distinct model families get tried, since it is the axis that permits or forbids trying more than one family in the first place; that the output-format and emphasis axes tie with it suggests that being asked to reason at length, or to explore broadly, also nudges the agent toward trying both families rather than one, even though neither axis mentions model families directly. The number of experiments run is split evenly between the stopping-rule axis, which sets the budget in the first place, and the emphasis axis, which affects how quickly that budget gets used up by determining whether early experiments tend to keep improving or not.

The held-out configuration used for causal validation – narrow model family, an unspecified evaluation metric, a fixed budget of exactly three experiments, terse output, and immediate-refinement emphasis – was deliberately excluded from the eight configurations used to fit the model above. Table 3 compares what the fitted model predicted for two of its behavioral quantities, before this configuration was ever executed, against what was actually observed once it was run twice with two fresh random seeds.

Table 3: Predicted versus observed behavior for the held-out configuration.

Behavioral quantity	Predicted (before running)	Observed (after running)	Difference
Number of experiments	3.25	3.00	−0.25
Best accuracy reached	0.9648	0.9610	−0.0038

The number-of-experiments prediction is essentially exact: the model correctly identified that this configuration’s fixed budget of three experiments would dominate, and predicted a value within a quarter of an experiment of what actually happened. The accuracy prediction is off by about 0.004, in the direction of overestimating how much the immediate-refinement strategy helps. This is not a random miss; it has a specific explanation. Because the eight-configuration design used in Step 5 cannot separate interactions between axes, the estimated benefit of immediate refinement was necessarily averaged across configurations that also varied the breadth and stopping-rule axes. If the true benefit of immediate refinement actually depends on, say, whether the agent is also allowed to compare two model families, then a model that treats it as a fixed, additive bonus will get some configurations slightly wrong – which is exactly what happened here. Read this way, the miss is not a failure of the method; a wrong-but-explainable prediction that points at a specific, checkable gap (an interaction the design could not see) is a more useful outcome than either a prediction that always happens to be right, or a validation step that is skipped altogether.

6 Limitations

The most significant limitation of this pilot concerns how the agent’s decisions were actually produced. Genuinely independent behavior – decisions made by a language model that has seen nothing but the instruction file for that specific run, with no memory of any other run – requires a way to make many separate, freshly-started calls to such a model. No such access was available while this pilot was carried out. In its place, the author personally played the role of the agent for every one of the eighteen runs, reading each generated instruction file and trying to act on only what it said. This is a genuine confound: the author already knew, throughout, that this was a controlled study and roughly what each axis was meant to test, which a language model given only the instruction file would not know. The results in Section 5 should therefore be read as a demonstration that the analysis pipeline itself works correctly and can surface plausible, self-consistent effects, not as an established fact about how an independent language model behaves under these instructions. Replacing this substitution with real, independent calls to a language model, and checking whether the same effects reappear, is the single most important next step for this work.

A second limitation is that no human reference trace exists yet, for the reasons given in Section 3: it requires an actual recorded session, which was set aside for this pilot. One consequence is that a quantity the original plan calls for – how closely an automated run’s research process resembles a human’s – has not yet been computed, though the logging format used throughout was chosen so that adding it later will not require re-running anything already done.

A third limitation is scale. Two repeats per configuration were used rather than the three to five that would more reliably separate a genuine configuration effect from ordinary noise, and eight configurations support only individual-axis effects, not interactions between pairs of axes – a limitation the causal-validation result above shows is not merely theoretical. A fourth and final limitation is scope: every experiment concerns one small dataset and two model families with a modest set of hyperparameters, so the specific numbers reported here should not be assumed to generalize beyond this setting without repeating the study elsewhere.

7 Conclusion

This pilot carried out five of six planned steps toward explaining, rather than merely observing, how an instruction file shapes the behavior of an automated research loop, by treating the instruction file the same way automated machine learning treats a hyperparameter vector: as a small set of discrete choices whose individual effect on behavior can be measured with a fractional factorial design, summarized with a simple interpretable model, and checked with a real held-out intervention. Every number in this report came from actually executed machine learning experiments.

The method recovered an expected relationship cleanly (the instruction to write full explanations is the dominant driver of how much text the agent produces) and surfaced a less obvious one (whether the agent is told to explore broadly or refine immediately turns out to be the strongest single predictor of the accuracy it eventually reaches). Its held-out validation step produced a genuinely informative result: one quantity predicted almost exactly, and one prediction that missed by a small amount in a way that points precisely at what the design could not see – an interaction between the emphasis axis and one of the others.

The most useful next steps, in order, are: replacing the author’s own role as the acting agent with independent calls to a language model, to check whether the effects found here survive that change; recording the human reference trace the original plan calls for; expanding the ablation design so that interactions between axes, in particular those involving the emphasis axis, can be estimated directly rather than only inferred from a validation miss; and repeating the study on more than one dataset to see how much of what was found here is specific to this particular benchmark.

References

- [1] C. Fawcett and H. H. Hoos. Analysing differences between algorithm configurations through ablation. *Journal of Heuristics*, 22(4):431–458, 2016.
- [2] F. Hutter, H. Hoos, and K. Leyton-Brown. An efficient approach for assessing hyperparameter importance. *Proceedings of the 31st International Conference on Machine Learning (ICML)*, 2014.
- [3] R. L. Plackett and J. P. Burman. The design of optimum multifactorial experiments. *Biometrika*, 33(4):305–325, 1946.